



An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.

[Follow publication](#)

MobileNetV2: Inverted Residuals and Linear Bottlenecks



Paul-Louis Pröve

Follow

5 min read · Apr 11, 2018



9

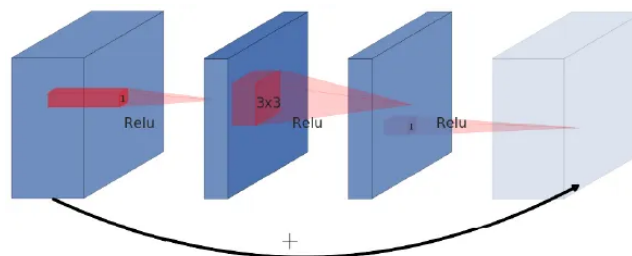


In April 2017 a group of researchers from Google published a paper which introduced a neural network architecture that was optimized for mobile devices. They strived for a model that delivered high accuracy while keeping the parameters and mathematical operations as low as possible. This was much needed in order to bring deep neural networks to smartphones.

The architecture dubbed MobileNet revolves around the idea of using depthwise separable convolutions, which consist of a depthwise and a pointwise convolution after one another. If you're a little fuzzy on the details of this operation feel free to check out my other [article](#) that explains this concept in detail. MobileNetV2 extends its predecessor with 2 main ideas.

Inverted Residuals

Residual blocks connect the beginning and end of a convolutional block with a skip connection. By adding these two states the network has the opportunity of accessing earlier activations that weren't modified in the convolutional block. This approach turned out to be essential in order to build networks of great depth.



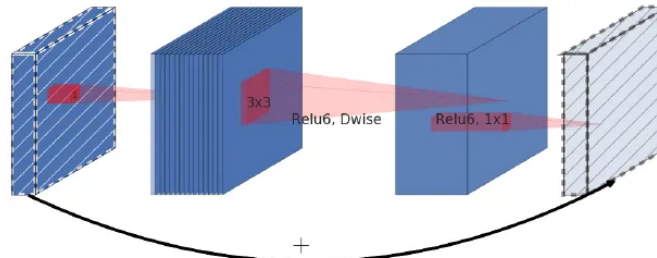
A residual block connects wide layers with a skip connection while layers in between are narrow

When looking a little closer at the skip connection we notice that an original residual block follows a wide->narrow->wide approach concerning the number of channels. The input has a high number of channels, which are compressed with an inexpensive 1x1 convolution. That way the following 3x3 convolution has far fewer parameters. In order to add input and output in the end the number of channels is increased again using another 1x1 convolution. In Keras it would look like this:

```
def residual_block(x, squeeze=16, expand=64):  
    m = Conv2D(squeeze, (1,1), activation='relu')(x)  
    m = Conv2D(squeeze, (3,3), activation='relu')(m)
```

```
m = Conv2D(expand, (1,1), activation='relu')(m)
return Add()([m, x])
```

On the other hand, MobileNetV2 follows a narrow->wide->narrow approach. The first step widens the network using a 1x1 convolution because the following 3x3 depthwise convolution already greatly reduces the number of parameters. Afterwards another 1x1 convolution squeezes the network in order to match the initial number of channels.



An inverted residual block connects narrow layers with a skip connection while layers in between are wide

In Keras it would look like this:

```
def inverted_residual_block(x, expand=64, squeeze=16):
    m = Conv2D(expand, (1,1), activation='relu')(x)
    m = DepthwiseConv2D((3,3), activation='relu')(m)
    m = Conv2D(squeeze, (1,1), activation='relu')(m)
    return Add()([m, x])
```

The authors describe this idea as an inverted residual block because skip connections exist between narrow parts of the network which is opposite of how an original residual connection works. When you run the two snippets above you will notice that the inverted block has far fewer parameters.

Linear Bottlenecks

The reason we use non-linear activation functions in neural networks is that multiple matrix multiplications cannot be reduced to a single numerical operation. It allows us to build neural networks that have multiple layers. At the same time the activation function ReLU, which is commonly used in neural networks, discards values that are smaller than 0. This loss of information can be tackled by increasing the number of channels in order to increase the capacity of the network.

Get Paul-Louis Pröve's stories in your inbox

Join Medium for free to get updates from this writer.

☒ Remember me for faster sign in

With inverted residual blocks we do the opposite and squeeze the layers where the skip connections are linked. This hurts the performance of the network. The authors introduced the idea of a linear bottleneck where the last convolution of a residual block has a linear output before it's added to the initial activations. Putting this into code is super simple as we simply discard the last activation function of the convolutional block:

```
def inverted_linear_residual_block(x, expand=64, squeeze=16):
    m = Conv2D(expand, (1,1), activation='relu')(x)
    m = DepthwiseConv2D((3,3), activation='relu')(m)
    m = Conv2D(squeeze, (1,1))(m)
    return Add()([m, x])
```

ReLU6

The snippet above shows the structure of a convolutional block that incorporates inverted residuals and linear bottlenecks. If you want to match MobileNetV2 as closely as possible there are two other pieces you need. The first aspect simply adds Batch Normalization behind every convolutional layer as you're probably used to by now anyways.

The second addition is not quite as common. The authors use ReLU6 instead of ReLU, which limits the value of activations to a maximum of...well...6. The activation is linear as long as it's between 0 and 6.

```
def relu(x):
    return max(0, x)

def relu6(x):
    return min(max(0, x), 6)
```

This is helpful when you're dealing with fixed point inference. It limits the information left of the decimal point to 3 bits, meaning we have a guaranteed precision right of the decimal point. This was also used in the original MobileNet paper. The final building block looks like this:

```
def bottleneck_block(x, expand=64, squeeze=16):
    m = Conv2D(expand, (1,1))(x)
    m = BatchNormalization()(m)
    m = Activation('relu6')(m)
    m = DepthwiseConv2D((3,3))(m)
    m = BatchNormalization()(m)
    m = Activation('relu6')(m)
    m = Conv2D(squeeze, (1,1))(m)
    m = BatchNormalization()(m)
    return Add()([m, x])
```

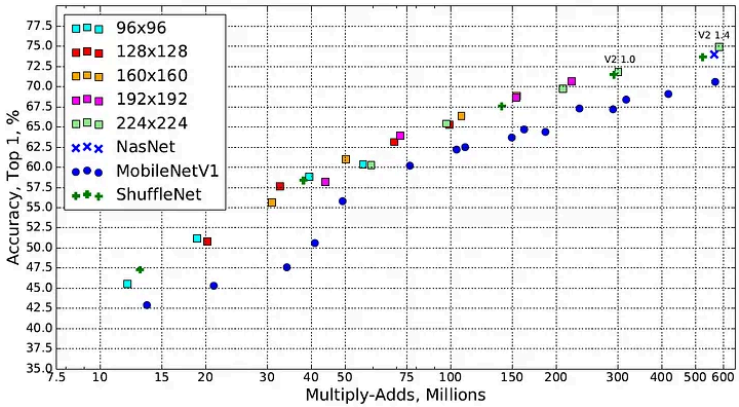
The Architecture

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

The MobileNetV2 architecture

Now that we understand the building block of MobileNetV2 we can take a look at the entire architecture. In the table you can see how the bottleneck blocks are arranged. t stands for expansion rate of the channels. As you can

see they used a factor of 6 opposed to the 4 in our example. c represents the number of input channels and n how often the block is repeated. Lastly s tells us whether the first repetition of a block used a stride of 2 for the downsampling process. All in all it's a very simple and common assembly of convolutional blocks.



Performance of MobileNetV2 and other architectures on ImageNet

Concluding Thoughts

Something that I'm particularly happy about is the fact that MobileNetV2 provides a similar parameter efficiency to NASNet. NASNet is the current state of the art on several image recognition tasks. It's building blocks are quite complex making it rather unintuitive why it works so well. The building block of NASNet were not designed by humans but by another neural network. Introducing a simple architecture like MobileNetV2 that shows a comparable efficiency makes me a little more confident that the next big architecture might be designed by a human being as well.

Machine Learning

Deep Learning

Neural Networks

Google

Convolutional Network

9

Published in TDS Archive

828K followers · Last published Feb 3, 2025

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.

Written by Paul-Louis Pröve

1.1K followers · 189 following

AI @ Tensora - We are hiring!

Follow

Follow

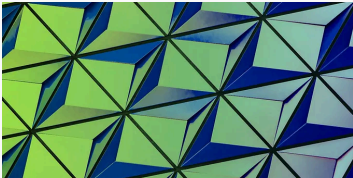
Responses (9)

Write a response

What are your thoughts?

See all responses

More from Paul-Louis Pröve and TDS Archive



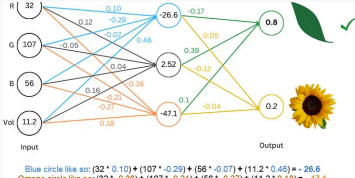
In TDS Archive by Paul-Louis Pröve

An Introduction to different Types of Convolutions in Deep Learning

Let me give you a quick overview of different types of convolutions and what their benefit...

Jul 22, 2017

31



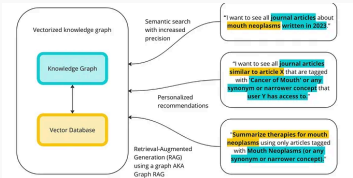
In TDS Archive by Rohit Patel

Understanding LLMs from Scratch Using Middle School Math

In this article, we talk about how LLMs work, from scratch—assuming only that you know...

Oct 19, 2024

106



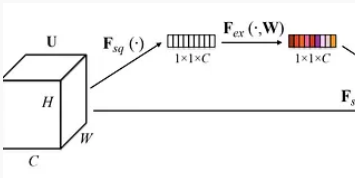
In TDS Archive by Steve Hedden

How to Implement Graph RAG Using Knowledge Graphs and...

A Step-by-Step Tutorial on Implementing Retrieval-Augmented Generation (RAG),...

Sep 6, 2024

20



In TDS Archive by Paul-Louis Pröve

Squeeze-and-Excitation Networks

Setting a new state of the art on ImageNet

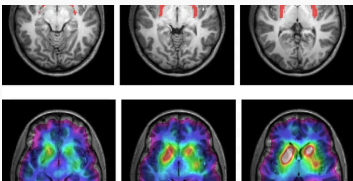
Oct 17, 2017

8

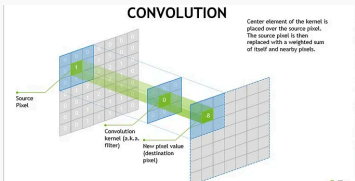
See all from Paul-Louis Pröve

See all from TDS Archive

Recommended from Medium



In Write A Catalyst by Dr. Patricia Schmidt



In Towards AI by Ampatishan Sivalingam

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

Jan 14 938



In ILLUMINATION by Sufyan Maan, M.Eng

I Woke Up at 4:30 AM Every Day for 30 Days — Here Is What Nobody...

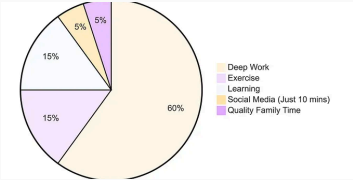
Here is what actually happened, from someone who did it & tracked everything.

Apr 4 326

The Death of CNNs: How Vision Transformers Rewrote Computer...

From AlexNet's 2012 revolution to ResNet's dominance, and why it all became obsolete...

Feb 6 9

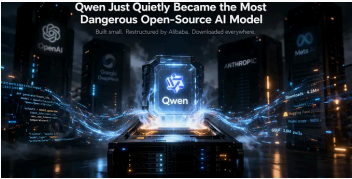


In Level Up Coding by Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brai...

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who...

Dec 28, 2025 490



Suleiman Tawil

Qwen Just Quietly Became the Most Dangerous Open-Source AI...

The most-downloaded AI model family on Earth was built by a small team with fewer...

Mar 31 44



In E²—Entertain Enlighten Emp... by Joshua Mas...

How to Spot a Liar in 3 Questions (Without Them Knowing)

The secret to detecting deception isn't about body language—it's about "Cognitive Load."

Dec 11, 2025 527

See more recommendations